计算机科学与技术学院　大数据管理与分析　课程实验报告

实验题目：熟悉常用的 HBase 操作与 PageRank 算法的 MapReduce / Spark 实现		学号：
日期：2026.11.18	班级：	姓名：
Email：		

实验目的：

（1）理解 HBase 在 Hadoop 体系结构中的角色，掌握其数据模型（行键、列族、列限定符、时间戳、Cell）以及“按行键有序、按列族存储、稀疏、多版本”的特点。

（2）熟练使用 HBase Shell 完成列出表信息、显示表的全部记录、增加与删除列族或列、清空表、统计行数 5 项操作，并能用 Java API 编程实现完全相同的功能。

（3）理解 PageRank 的随机游走模型与迭代公式，弄清阻尼系数、悬挂页面（无出链的页面）以及收敛速度三者的关系。

（4）分别用 MapReduce 和 Spark 实现 10 轮迭代的 PageRank，把结果格式化为“(网页，PR 值)”，并比较两种框架在“迭代型作业”上保存中间状态的差异。

实验软件和硬件环境：

硬件：x86_64 虚拟机（VMware Workstation 17），4 核 CPU、8 GB 内存（Hadoop、HBase、Spark 同时运行，内存比实验一调大一倍）、60 GB 磁盘；宿主操作系统 Windows 11。

软件：Ubuntu 20.04.6 LTS（64 位）；OpenJDK 1.8.0_362；Hadoop 3.3.4 伪分布式（`fs.defaultFS = hdfs://localhost:9000`，副本数 1）；HBase 2.4.14 伪分布式（`hbase.rootdir = hdfs://localhost:9000/hbase`，自带 ZooKeeper）；Spark 3.3.2（`spark.master = local[2]`）；Maven 3.8.6（`hadoop-client`、`hbase-client`、`spark-core` 均为 provided）；Python 3.8 + PySpark 3.3.2。

实验原理和方法：

（1）HBase 的角色与数据模型。HBase 是构建在 HDFS 之上的列族式（column-family）分布式数据库：底层用 HDFS 存放数据文件，对外提供按行键的随机读写，并向 MapReduce / Spark 提供表输入输出格式。一张表按行键的字典序划分为若干 Region，由 RegionServer 负责，HMaster 负责元数据与负载均衡。数据模型可概括为 `Table -> RowKey -> ColumnFamily -> Qualifier -> (Value, Timestamp)`：同一个列族的列在物理上存放在一起，列限定符不需要预先定义，写入时即可动态增加；行键决定了一条记录的物理位置，也决定了 `scan` 的范围查询是否高效。

（2）关系表到 HBase 表的转换。原则是“一张关系表对应一张 HBase 表，主键作行键，经常一起访问的属性放同一个列族”。本实验的三张表设计为：student（行键 S_No，列族 Info 存 name/sex/age，列族 Score 存各门课成绩，列限定符直接用课程名）、course（行键 C_No，列族 Info 存 name/credit）、sc（行键 S_No_C_No，列族 Score）。把成绩放进 Score 列族后，新增一门选修课只需要 put 一个新列限定符，不需要像关系表那样改表结构或加行，这正是列族模型的优势。

（3）PageRank 的原理。把网页看作结点、超链接看作有向边，PageRank 是随机游走模型的平稳分布：浏览者以概率 d 沿当前页的出链继续点击，以概率 (1-d) 随机跳到任意页面。结点 p 的 PR 值为

PR(p) = (1-d)/N + d · (Σ_{q→p} PR(q)/outDegree(q) + PR_dangling/N)

其中 N 为页面总数，d 取 0.85，PR_dangling 是所有悬挂页面的 PR 之和。写成矩阵形式是 $PR = (1-d)/N \cdot 1 + d \cdot M^{T} \cdot PR$，M 为列随机矩阵；求解即幂迭代 $PR^{(k)} = (1-d)/N + d \cdot M^{T} \cdot PR^{(k-1)}$。初始值取 $PR^{(0)}(p) = 1/N$，则每轮迭代后 PR 之和恒为 1（概率质量守恒），不同规模的图之间结果可直接比较。

（4）悬挂页面与概率漏出。没有出链的页面在 M 中对应整列为 0，此时 M 不是随机矩阵，直接相乘会让概率质量不断流失，PR 之和逐轮下降；因此在 map 阶段把这些页面的 PR 平均分给所有页面（等价于让它以等概率指向任何页面），迭代过程才严格保持总和为 1。

（5）MapReduce 的迭代设计。MapReduce 作业之间不能共享内存，第 k 轮的 PR 结果必须先落盘到 HDFS，第 k+1 轮作业再通过分布式缓存（DistributedCache）把它读进每个 map 任务的内存 HashMap。Mapper 的输入仍是原始的“网页 + 出链集合”，因此一次作业就能完成“分发 + 聚集”，不需要课本上分两个作业写的 Distribute 和 Aggregate。一轮迭代结束后，reducer 输出 网页 \t 新 PR，即成为下一轮的缓存文件；共 1 个初始化作业 + 10 个迭代作业 + 1 个排序格式化作业。

（6）Spark 的迭代设计。Spark 把图数据和 PR 向量都存成 RDD 并缓存在内存里，一轮迭代就是 links.join(ranks) → flatMap 出发放量 → reduceByKey 累加 → mapValues 套公式 这几步的血统（lineage）增长，中间结果不落盘；只在最后按 PR 降序排序输出。两者用同一个公式，因此结果应当逐位相同。

实验步骤：（不要求罗列完整源代码）

一、HBase 的常用 Shell 操作

启动 start-hbase.sh 后用 jps 可看到 HMaster 与 HRegionServer 两个进程，再执行 hbase shell 进入交互命令行。按 5 项任务依次操作（脚本 scripts/01_hbase_ops.sh），其中建表时同时创建了实验要求的三张表：

```
hbase(main):001:0> list
TABLE
course       sc       student
3 row(s)
```

```
hbase(main):002:0> scan 'student'
ROW        COLUMN+CELL
 2015001   column=Info:age,  timestamp=1763..., value=23
 2015001   column=Info:name, timestamp=1763..., value=Zhangsan
 2015002   column=Info:name, timestamp=1763..., value=Mary
 2015003   column=Info:name, timestamp=1763..., value=Lisi
...

hbase(main):003:0> alter 'student', NAME => 'Email'          # 增加列族
hbase(main):004:0> describe 'student'                        # 可见 Info/Score/Email
hbase(main):005:0> delete 'student', '2015002', 'Score:English'   # 删除某列
hbase(main):006:0> alter 'student', NAME => 'Email', METHOD => 'delete'  # 删除列族
hbase(main):007:0> count 'student'
3 row(s)
hbase(main):008:0> truncate 'sc'                             # 清空表（结构保留）
hbase(main):009:0> count 'sc'
0 row(s)
```

说明：scan 会把每一行展开成“列族:列 = 值 + 时间戳”，列的顺序按字典序排列；alter ... METHOD => 'delete' 删除列族时该列族的全部数据一并消失；truncate 等价于 disable + drop + create，执行后表结构还在、行数为 0。

二、HBase 的 Java API 与关系表的转换

程序 HBaseApp 用子命令的形式实现全部功能，统一通过 ConnectionFactory.createConnection(HBaseConfiguration.create()) 取得 Connection，再 getTable/getAdmin 操作表与元数据。5 项通用功能与 Shell 命令一一对应：list 对 admin.listTableNames() 加 getDescriptor（可同时得到列族与行数）；scan 对 Table.getScanner(new Scan())；增加/删除列族对 admin.addColumnFamily 与 admin.deleteColumnFamily，删除某列对 Delete.addColumns(family, qualifier)；清空表对 disableTable + truncateTable；统计行数遍历 ResultScanner 计数。

按实验要求把三张关系表转换为 HBase 表后，createTable(tableName, fields) 先判断表是否存在（存在则 disable + delete），再为 fields 中的每个列族调用 TableDescriptorBuilder.setColumnFamily；addRecord 把 fields 中的 列族:列 拆开，组装成一个 Put 批量写入；scanColumn 判断参数里有没有冒号来决定是列出整个列族还是单列，取不到值时输出 null；modifyData 直接 put 覆盖（HBase 以时间戳区分版本，新写入即为最新值）；deleteRow 用 Delete(row) 删除整行。执行效果如下：

```
$ hadoop jar target/bigdata-lab3.jar cn.sdu.bigdata.lab3.hbase.HBaseApp init
已创建表 student，列族 = Info,Score       （course、sc 同）
$ ... HBaseApp showcol student Score:Computer Science
--- student 的列 Score:Computer Science ---
2015001      null
2015002      77
2015003      95
$ ... HBaseApp modify student 2015001 Score:Math 95
已修改 2015001 的 Score:Math：86 -> 95
$ ... HBaseApp delRow sc 2015002_123003
```

```
已删除 sc 中行 2015002_123003
```

三、PageRank 的 MapReduce 实现

数据集 data/pagerank.txt 共 25 个页面、83 条链接，每行是“网页 + 制表符 + 出链集合”，其中 hadoop_tutorial_pdf 只有入链没有出链，用来检验悬挂页面的处理。核心的 map / reduce 逻辑如下（完整代码见 PageRankMR.java）：

```
// map：把当前页面的 PR 按出链数平均分给它指向的页面
double pr = prValues.get(page);              // prValues 来自分布式缓存
DoubleWritable share;
if (links.isEmpty()) {                       // 悬挂页面：PR 平摊给所有页面
    share = new DoubleWritable(pr / nPages);
    for (String p : allPages) context.write(new Text(p), share);
} else {
    share = new DoubleWritable(pr / links.size());
    for (String link : links) context.write(new Text(link), share);
}
// reduce：同一个网页的所有入链贡献求和后套用公式
double newPr = (1 - DAMPING) / nPages + DAMPING * sum;
```

运行命令为 hadoop jar bigdata-lab3.jar ...PageRankMR <输入路径> <输出目录> 10，控制台会打印 12 次作业的进度以及每轮耗时。10 轮迭代后由收尾作业统一按 PR 降序输出，25 个页面全部参与计算，前 10 名如下（与要求的格式一致，每行一组）：

```
(Apache_Hadoop,0.0927489200)
(Distributed_computing,0.0779326630)
(MapReduce,0.0667616227)
(Big_data,0.0639704934)
(Machine_learning,0.0627814603)
(Data_mining,0.0597310331)
(Database,0.0563539254)
(Cloud_computing,0.0529207550)
(NoSQL,0.0460786367)
(Computer_cluster,0.0412556136)
```

其余 15 个页面（如 Apache_Spark 0.0403102880、HBase 0.0309778503、HDFS 0.0266070203）的结果见仓库 out/pagerank_result.txt。结果符合预期：Apache_Hadoop 以 0.0927489200 排第一，指向它的 Big_data、Apache_Spark、Hive、MapReduce 等 8 个页面本身排名也不低；唯一没有出链的 hadoop_tutorial_pdf 只被 Apache_Hadoop 指向、自己从不投票，PR 只有 0.0178719286；PageRank、Google 虽然与搜索引擎主题强相关，但在本数据集中只被少数页面指向，因而排在末尾（0.0153564384）。这说明 PR 衡量的是“被多少重要的页面指向”，而不是单纯的入链数量。

四、PageRank 的 Spark 实现与两种框架的对比

Spark 版本（spark/pagerank_spark.py，另有一份等价的 Java 版 PageRankSpark.java）把“图”和“PR 向量”都表示成 RDD，一轮迭代就是一次 join + 聚合：

```
links = sc.textFile(path).map(parse).cache()        # (网页, [出链...])
n = links.count();  d = 0.85
ranks = links.mapValues(lambda _: 1.0 / n)
```

```
for i in range(10):
    contrib = links.join(ranks).values().flatMap(
        lambda kv: [(t, kv[1] / len(kv[0])) for t in kv[0]])   # 分发 PR
    dangling = (links.join(ranks).filter(lambda kv: not kv[1][0])
                     .values().map(lambda kv: kv[1]).sum())    # 悬挂页面质量
    ranks = (links.keys().map(lambda k: (k, 0.0)).union(contrib)
             .reduceByKey(lambda a, b: a + b)                  # 累加入链贡献
             .mapValues(lambda s: (1 - d) / n + d * dangling / n
                                  + d * s)).cache()
out = (ranks.map(lambda kv: (kv[1], kv[0])).sortByKey(False)
            .map(lambda kv: "(%s,%.10f)" % (kv[1], kv[0])).collect())
```

两个程序输出的 25 行结果完全一致（用 `diff` 比较无差异，最大数值差 1.4×10^{-17}，仅来自浮点加法顺序不同）。差别在运行方式：MapReduce 版本每一轮都要把 PR 写回 HDFS 再由下一轮的 map 读回，10 轮共 10 次落盘、10 次读盘，作业启动开销占了大头；Spark 版本把 `links` 和 `ranks` 缓存在内存中，10 轮迭代只是一个 DAG 内连续触发 10 次计算，没有中间文件，因而同样 25 个页面的数据在 local[2] 下几乎是瞬间完成。

收敛情况：相邻两轮所有页面 PR 的最大变化量依次为 2.96e-02、1.79e-02、7.55e-03、5.53e-03、2.12e-03、2.01e-03、7.67e-04、7.79e-04、2.86e-04，按大约每轮缩小到 0.72 倍的规律衰减（$0.85^2 = 0.7225$），第 10 轮仍有 2.9×10^{-4} 的残差，说明 10 轮迭代已经十分接近但尚未完全收敛；这与阻尼因子决定的收敛速度 d^k 一致。

以上两部分程序与全部脚本已整理为 Maven 工程，上传至 GitHub 公开仓库：`https://github.com/YuuAnnooo/sdu-bigdata-labs`（Public，本实验位于 `bigdata-lab3/`）。目录约定：`src/main/java` 放 Java 源码（`hbase/HBaseApp`、`pagerank/PageRankMR`、`pagerank/PageRankSpark`），`spark/` 放 PySpark 版，`data/` 放数据集，`out/` 放输出结果，`tools/` 放数据集生成与本地校验脚本；`mvn -q clean package` 生成 `target/bigdata-lab3.jar`。

结论分析与体会：

（1）HBase 部分。用 Shell 敲一遍之后才体会到“列族”是 HBase 真正的物理单位：`alter` 增加列族、`delete` 删除某列、`truncate` 清空数据都很快，但一旦涉及列族就要重建 Region；而列限定符可以随写随有，新增一门课只要 `put` 一个新列，这与关系表必须改表结构形成鲜明对比，也解释了为什么 HBase 适合“字段经常增加、按行键查询”的场景。反过来，如果业务需要按成绩排序或跨行统计，就必须自己建索引表或走 MapReduce/Spark 全表扫描，这也正是 HBase 不能完全替代关系数据库的原因。

（2）PageRank 部分。它的本质是一个迭代求矩阵主特征向量的问题，而迭代恰好是 MapReduce 最不擅长的形态——每轮都要落盘、每轮都要重新启动任务，开销与轮数成正比。改用 Spark 后中间结果留在内存里，代码也从“一个作业一个类”缩减为一次 join 加一次聚合，这直观说明了为什么迭代型算法（PageRank、

K-Means、逻辑回归）更适合 Spark。

（3）实现中的几个细节。① 初始值取 1/N 而不是 1，才能保证每轮 PR 之和为 1，否则数值会随图规模漂移；② 每个页面在 map 端给自己发一个 0 贡献，是为了让“没有任何入链”的页面也能出现在下一轮结果里，否则它会从结果文件中消失；③ 悬挂页面如果不处理，每轮都会漏掉一部分概率质量，PR 之和会肉眼可见地逐轮下降，这是调试时最容易忽略的 bug；④ 排序输出必须把 `setNumReduceTasks` 设为 1，或者使用全序分区，否则只能保证分区内有序。

（4）收获。这次实验把 HBase 与 PageRank 放在一起，恰好覆盖了“存储”和“计算”两个侧面：HBase 解决按行键快速读写与稀疏数据的存储问题，PageRank 则展示了分布式框架上迭代计算的两种典型实现路径。把代码整理成 Maven 工程、把每一步写成可重复执行的脚本，再用一个不依赖集群的本地校验脚本核对算法逻辑（`tools/local_check.py` 用纯 Python 分别复现 MapReduce 和 Spark 两种语义并断言输出一致），比在集群上反复试错效率高得多。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：

问题一：HBase 的行键该如何设计？用“时间戳 + 用户 ID”作行键会怎样？

答：行键决定了记录在 Region 中的位置（按字典序排列），也决定了 `scan` 能否用上范围查询，所以要选择“查询最常用的条件 + 分布均匀”的字段。若用递增的时间戳作行键前缀，所有新写入都会落在同一个 Region 上（写热点），负载无法分摊，Region 还会持续分裂和 Compaction，写入性能急剧下降。解决办法是打散前缀：加盐（`salt = hash(rowkey) % N` 放在行键前面）、哈希（直接用 MD5 或取模后的值）、或反转时间戳（`Long.MAX_VALUE - ts`，既能打散又能让最新数据排在最前面）。代价是这些行键无法直接按业务顺序范围扫描，通常要另建一张索引表来换取查询能力。

问题二：MapReduce 版的 PageRank 为什么每轮都要把状态写回 HDFS？

答：MapReduce 的编程模型规定 reducer 输出到 HDFS，作业之间没有任何共享状态，下一个作业的 map 任务运行在不同机器、不同 JVM 上，只能通过文件系统交换数据；所以必须“第 k 轮输出 → 第 k+1 轮用分布式缓存读入”，这也是它在迭代计算上慢的根本原因。要在 MapReduce 里“放内存”只能借助外部机制：把 PR 向量写进 HBase 或 Redis 之类的存储，或者在 Driver 中把一个 `job.waitForCompletion` 循环写成立式内存传递——但只要还是多个作业，就无法避免作业启动和 shuffle 的开销。真正的解法是改用 Spark 这类以 RDD/DAG 为核心的框架：`ranks` 缓存后，迭代只是同一份血统上的重复计算，不需要落盘。

问题三：迭代 10 轮后 PR 值仍在变化，迭代次数越多越好吗？

答：幂迭代的误差按 d^k 衰减，$d = 0.85$ 时 10 轮后仍有约 $0.85^{10} \approx 0.20$ 的残差，本实验实测相邻两轮最大变化从 3.0×10^{-2} 降到 2.9×10^{-4}，正是这个规

律；若要求 10^{-6} 的精度，大约需要迭代 60 轮。另一方面，收敛是否单调取决于图的结构：本数据集是明显的二部结构（概念页之间互相链接、很少出现单向链），对应矩阵的特征值有一对接近 -1 的共轭复根，PR 会在两个值之间来回摆动，实测相邻两轮的差按 $0.72 \approx d^2$ 衰减，而不是按 d 衰减，表现为“偶数轮接近、奇数轮偏离”。只要 $d < 1$，振荡最终一定收敛；此时可以取相邻两轮的平均值、或采用 Aitken 加速来减少迭代次数。这也解释了为什么阻尼因子不能取 1——$d = 1$ 时随机游走可能不收敛，而且与网页链接的真实浏览行为也不符。